FIRST ROBOTICS COMPETITION

REEFSCAPE℠ PRESENTED BY HAAS Gene Haas Foundation

FIRST® DIVE℠ presented by Qualcomm

firstinspires.org/robotics/frc

2025 *FIRST*® Robotics Competition

KitBot C++ Software Guide

Rev. Jan 4, 2025

1 Contents

2 Document Overview

This document will take you through how to get your 2025 KitBot up and running using the provided C++ example code. To avoid content duplication this document frequently links to WPILib documentation for accomplishing specific steps along the way. In addition to getting you up and running with the provided code, this document will walk through the structure of that code so you can understand how it operates. Finally, we'll walk through some of the most likely changes you may wish to make to the code and provide concrete examples of how to make those modifications.

To get started with the example code, or to make some of the modifications described, minimal understanding of C++ is required. The code and modification examples provided will likely provide enough of a pattern to get you going. To understand the walkthrough, or to make modifications not described in this document, a more thorough understanding of C++ is likely required. C++ can be a bit daunting for beginners if you don't have support. If your team doesn't have mentors familiar with C++, Java or Python may be better choices for programming your robot. With that said, the WPILib Docs have a couple options if you are looking to learn or improve your C++ knowledge.

This document, and the provided example code, assumes the use of the SPARK MAX controllers provided in the rookie Kickoff Kit.

3 Getting Started with your KitBot code

3.1 Wiring your robot

Use the WPILib Zero-to-Robot wiring document to help you get your robot wired up. The KitBot wiring and code is documented with the Control System components that have recently come in the Rookie Kit of Parts (i.e. REV PDH and Spark MAX controllers), the KitBot wiring and code can be adapted to other electronics but that adaptation is not covered by these documents.

3.2 Configuring hardware and development environment

Before you are able to load code and test out your robot, you will need to configure your hardware (roboRIO, radio, etc.) and get your development environment set up. Follow the WPILib Zero-to-Robot guide steps 2 through 4 to get everything set up and ensure you can deploy a basic robot project.

3.3 Opening the 2025 KitBot Example

The 2025 KitBot example code is provided in individual zip files for each language on the KitBot webpage. The C++ code contains two complete projects which illustrate different ways of creating Commands. Some description of the difference can be found in Ways of creating commands below. To open the C++ code:

1. Download and unzip the C++ example code. Make sure to unzip or copy to a permanent location, not in a temporary folder.
2. Open **WPILib VS Code** using the Start menu or desktop shortcuts
3. In the top left click **File->Open Folder** and browse to the "C++" folder inside of the unzipped example code, then open the desired one of the two example projects, and then click **Select Folder.**

3.4 Spark MAX firmware update and CAN IDs

Before using the SPARK MAXs with CAN control, they each need to be assigned a unique ID..

1. Install the REV Hardware Client
2. With the robot powered off, connect a USB cable between the computer and the SPARK MAX USB port. Leaving the robot powered off ensures only the single SPARK MAX is powered and avoids changing the IDs on unintended devices.
3. Update the firmware on the SPARK MAX
4. Set the CAN ID and Motor Type (you can skip the current limit) and save the settings
    a. CAN IDs for each device can be found in Constants.java. You can either set the devices to match these IDs, or set the IDs as desired (some teams set the CAN ID = the channel number the device is attached to on the PD) and then update these constants.
    b. Note: If you wish to "Spin the motor" as described on that page, make sure the robot is in a safe state to do so (wheels not touching the ground or table, all hands clear of the robot).

5. Repeat for all 5 devices on the robot. IF you've wired up the 6th Spark MAX you likely want to set it to a non-conflicting ID as well.
6. While not required, if using the REV PDH you may wish to check that it has the latest firmware at this time as well. Do not change the ID of the PDH off of the default, each device type has a separate ID space and your PDH will not conflict with your SPARK MAX even if set to the same ID.

Now that all your devices are configured, you can do a preliminary check that your CAN bus is wired properly using the REV Hardware client. While plugged into any REV device on your CAN bus with a USB cable, power on the robot and you should see all the other devices listed in the left pane of the REV Hardware Client, under the CAN Bus heading. If you don't see all of the devices, you likely have one or more issues with your CAN bus wiring:

1. Verify that your CAN bus starts with the roboRIO and ends with a 120 ohm resistor, or the built in terminator of a Power Distribution Hub or Power Distribution Panel (with the termination set to On using the appropriate jumper or switch).
2. Check that your CAN bus connections all match yellow-yellow and green-green.
3. Check that all CAN wire connections are secure to each other and that the connectors are securely installed in each SPARK Max
4. If you're still having trouble, moving the USB connection around to different devices and seeing what each device can "see" on the bus can help pinpoint the location of an issue.

3.5 Installing REVLib

The software library for the SPARK MAX in CAN mode is provided by the vendor (REV Robotics). The 3rd party library configuration is already included in the project, but you will have to install the library itself. There are two ways you can do so:

1. **Recommended** - Install the library offline – This will ensure that the library persists on your machine even if you don't build new code for a while (online installations can be cleaned up automatically by Gradle).
    a. Download the latest version of REVLib using the link from the REV documentation.
    b. Unzip into the *C:\Users\Public\wpilib\2025* directory on Windows and *~/wpilib/2025* directory on Unix-like systems.
2. Install Online - While the computer is connected to the Internet, click the WPILib icon in the top right of the VSCode window to bring up the WPILib extension prompt, then start typing "Build Robot Code" and select that option when it appears. Because the library configuration is already included in the project, this will automatically fetch the library from the internet.

To install additional Vendor libraries you can use the WPILib Vendor Library manager inside VS Code.

3.6 Deploying and testing the KitBot Example

To deploy the example to your robot, you will need to set the Team Number on the project. Click the **WPILib icon** in the top right corner of the VS Code window (W inside a gear) to open the WPILib prompt and start typing "Set Team Number" and select that option when it appears. Enter your team number (no leading 0s – e.g. 123 or 9996) and press Enter.

You are now ready to deploy the KitBot example just like you deployed the test project in Step 4 of the Zero-to-Robot guide.

Warning: Make sure you have space in all directions when operating a robot. Even with known code, the robot may move with unexpected speed or in unexpected directions. Be prepared to Disable (Enter) or E-stop (Spacebar) the robot if necessary. The 2025 KitBot code contains a very simple autonomous routine that will move the robot forwards at ½ speed for 1 second when the robot is enabled in Autonomous mode.

3.7 Configuring Gamepads

The code is set up to use the Xbox controller class. The Logitech F310 gamepads provided in the Kit of Parts will appear like Xbox controllers to the WPILib software if they are configured in the correct mode. To set up the controllers, check that the switch on the back of the controller is set the 'X' setting. Then when using the controller, make sure the LED next to the Mode button is off, if it is on press the Mode button to toggle it. When the Mode button is on, the controller swaps the function of the left Analog stick and the D-pad.

3.8 What does the code do?

The provided code implements the following robot controls in Teleoperated:

- Driver controller is an Xbox Controller in Slot 0 of the Driver Station
    - Controls the robot drivetrain using Split-stick Arcade Drive
        - Y-axis (vertical) of left stick controls forward-back movement of drivetrain
        - X-axis (horizontal) of right stick controls rotation of drivetrain
- Operator controller is an Xbox controller in Slot 1 of the Driver Station
    - Controls the gamepiece roller using the triggers and buttons
        - Left Trigger – Runs the roller motor inward (tries to push the Coral back up the ramp) at variable speed.
        - Right Trigger – Runs the roller motor outward (tries to eject the Coral) at variable speed.
        - A-Button – Runs the roller motor outward at specific power.

Because of the way the code is implemented, it is not recommended to press the triggers at the same time as the behavior will be difficult to predict. Pressing the left and right triggers at the same time will add their values together. For example, pressing the left trigger down halfway, and the right trigger down all the way, the result will be the motor spinning outward at ½ speed.

4 Overall Code Structure

The provided code utilizes the Command-Based programming structure provided by WPILib. This structure breaks up the robot's actuators into "subsystems" which are controlled by "commands" or collections of commands (aptly name "command groups"). The Command-Based structure may be a bit overkill for a robot of this complexity, but it scales very well for teams looking to add additional functionality to their KitBot. Additionally, this code structure was used by over 60% of teams in 2024, increasing the likelihood that teams around you may be able to provide assistance before or during the event.

To read more about the Command-Based structure, see the Command-Based Programming chapter of the WPILib documentation.

4.1 Ways of creating commands

There are multiple ways that you can define commands within the Command-Based structure. This project uses two of these different types to provide exposure to what they would look like in a full robot project. The common ways of creating commands that are utilized in this project are:

- Traditional: Command/group is defined as its own class in its own file.
- Factory: Command/group is defined via a "Command Factory method" in the subsystem or a separate static command class.

Traditional	Factory
Generally easier to understand	**Slightly high learning curve**
Modularity – Commands can be long depending on the complexity	**Modularity – Commands are broken down into small pieces and strung together into groups**
Boilerplate – Command classes require subclassing and can be long	**Boilerplate – Commands are written as methods in the subsystem, meaning less unnecessary code**
Organization – Having many Command classes can cause clutter and slow programmer's efficiency	**Organization – Commands are grouped together based on the subsystems they require, leading to fewer/none dedicated Command classes**
Debugging – Easier to debug due to more defined structure and traditional logic	**Debugging – More difficult to debug due to the lambda functions and command compositions**

5 Code Walkthrough

5.1 Subsystems

As described in the What is Command-Based Programming article, "'subsystems' represent independently-controlled collections of robot hardware (such as motor controllers, sensors, pneumatic actuators, etc.) that operate together".

For the 2025 KitBot we have 5 motors that make up 2 subsystems, the Drivetrain, and the Roller. The 4 motors in the drivetrain always need to be working together to move the robot around the field and the Roller motor must spin to manipulate Coral.

Sometimes the boundaries between subsystems may not be so clear, if you have an arm with a shoulder and wrist joint and a set of motorized wheels on the end, is that all one subsystem or multiple? The general rule of thumb to follow is think about what actions, or commands, you might have to control the subsystems. Do you think you might want the two pieces to be controlled independent of each other (i.e. run the intake in or out while moving the arm or wrist?). If you're unsure, err towards more smaller subsystems; you can always make commands that require multiple subsystems but if you end up wanting separate commands to control a single subsystem at the same time, you'll have to refactor the subsystem to split it up.

5.1.1 CANDriveSubsystem.h

This class is the subsystem for the drivetrain.

5.1.1.1 Includes

```
5   #pragma once
6
7   #include <Constants.h>
8   #include <frc/RobotController.h>
9   #include <frc/drive/DifferentialDrive.h>
10  #include <frc2/command/CommandPtr.h>
11  #include <frc2/command/SubsystemBase.h>
12  #include <rev/SparkMax.h>
```

This section starts with the line "#pragma once". This line tells the compiler to only define this class once, even if it is included from multiple different places (such as multiple commands that all use the same subsystem at different times). This is important to ensure that the code doesn't try to define our hardware multiple times which will result in errors.

The rest of this section consists of include statements, statements which tell the compiler what other code this code relies on. The majority of the includes here come from WPILib, these can be identified by the <> notation and the first part of the path being frc or frc2. The last include is from REVLib and is the header needed for using SPARK MAXes over CAN.

5.1.1.2 Class declaration, Member Variables and Constructor

```
14  class CANDriveSubsystem : public frc2::SubsystemBase {
15   public:
16    CANDriveSubsystem();
```

The first line of this image is the class declaration. This declares the name of our class and says that it's an extension of the SubsystemBase class. All subsystems should extend this class which provides some utility functions regarding setting the name of the subsystem, registering it with the scheduler, and sending information about it to the dashboard.

The next section declares the constructor. The actual implementation is found in the .cpp file.

5.1.1.3 Methods

```
18    void Periodic() override;
19
20    void SimulationPeriodic() override;
21
22    void ArcadeDrive(double xSpeed, double zRotation);
```

The next section declares the class methods, the implementations are found in the .cpp file. The Periodic and SimulationPeriodic methods are provided as part of the subsystem class, so they are annotated with "override" to indicate that we are overriding the base implementation. The ArcadeDrive method takes double type input parameters of speed and rotation values, matching the corresponding WPILib class we will be calling. The InlineCommands project will instead have a method that returns a CommandPtr which represents a Command that that handles the drive functionality.

5.1.1.4 Class Members

```
24   private:
25    rev::spark::SparkMax leftLeader;
26    rev::spark::SparkMax leftFollower;
27    rev::spark::SparkMax rightLeader;
28    rev::spark::SparkMax rightFollower;
29
30    frc::DifferentialDrive drive{leftLeader, rightLeader};
```

The last section is the class members. These are all included in the private section, the hardware should only be accessed directly by the subsystem methods, if command or other code need access to the hardware, we should write new subsystem methods to expose what the other code needs. The members of our drivetrain class are the 4 motor controllers, and the Differential Drive object that we will use to control the drivetrain as a single unit.

5.1.2 CANDriveSubsystem.cpp

5.1.2.1 Include and Using statements

```
5   #include <subsystems/CANDriveSubsystem.h>
6
7   using namespace DriveConstants;
```

The first section consists of include and using statements. There will typically be an include statement to include the .h file for the class the .cpp file belongs to, in many cases this will be the only include. By including our .h file, the .h includes are also able to be referenced here. Additional includes are only needed if the .cpp file references classes that are not referenced in the .h file.

The using statement allows us to use more compact syntax when referring to members of that namespace, in this case instead of having DriveConstants::LEFT_LEADER_ID, we are able to use just LEFT_LEADER_ID.

5.1.2.2 Class Constructor

```
9   // class to drive the robot over CAN
10  CANDriveSubsystem::CANDriveSubsystem()
11      : leftLeader{LEFT_LEADER_ID, rev::spark::SparkMax::MotorType::kBrushed},
12        leftFollower{LEFT_FOLLOWER_ID, rev::spark::SparkMax::MotorType::kBrushed},
13        rightLeader{RIGHT_LEADER_ID, rev::spark::SparkMax::MotorType::kBrushed},
14        rightFollower{RIGHT_FOLLOWER_ID, rev::spark::SparkMax::MotorType::kBrushed} {
15    // set can timeout, because this project only configures on startup and
16    // doesn't query any parameters, timeouts can be very long. Projects which
17    // update configuration or retrieve parameters during runtime should use much
18    // shorter timeouts such as the default
19    leftLeader.SetCANTimeout(250);
```

This next section of code is the constructor for the class. The first line indicates the class. The next set of lines after the colon is called an initializer list, it initializes the member variables of the class, in this case our motor controllers. For each controller we specify the ID and the motor type (CIM motors are brushed motors). The other class variables in this class have their initialization set in the .h file. The last part of the constructor is any additional methods to be called on construction. In this class, we need to configure the motor controllers, review the comments in the code for more information.

5.1.2.3 Methods

```
58  void CANDriveSubsystem::Periodic() {}
59
60  void CANDriveSubsystem::SimulationPeriodic() {}
61
62  // sets the speed of the drive motors
63  void CANDriveSubsystem::ArcadeDrive(double xSpeed, double zRotation) {
64    drive.ArcadeDrive(xSpeed, zRotation);
65  }
```

This section defines the implementation of the methods of the class. The Periodic method is part of the base Subsystem class and is called once per iteration of the scheduler (~every 20 ms). You could use this method to update sensor values, perform logging, or other tasks that you wish to occur regardless of what the subsystem is actually doing. In our simple drivetrain we don't have anything we wish to do periodically. The last method is the ArcadeDrive method, which calls the method of the same name on our drive object. In the Inline Commands project this last method is instead a Command factory that produces Command objects which handle the drive functionality.

5.1.3 CANRollerSubsystem.h

This class is the subsystem for the roller.

5.1.3.1 Includes, Class Declaration and Constructor

The first sections of this subsystem are very similar to the CANDriveSubsystem subsystem. See section 0 for more detailed description of each of these parts of the code.

5.1.3.2 Methods – Hardware Control

```
21    void RunRoller(double forward, double reverse);
```

The remainder of the subsystem class is hardware access methods. Here we define any methods that commands may need to call to get status from or perform actions on the subsystem. For our Roller this includes methods to set the speed of the axle. Again the Inline Command project has a Command factory method here instead.

5.1.3.3 Class Members

```
23  private:
24    rev::spark::SparkMax rollerMotor;
```

The KitBot Roller has one motor.

5.1.4 CANRollerSubsystem.cpp

5.1.4.1 Includes, Using Statement, Class Constructor

The first sections of this subsystem are very similar to the CANDriveSubsystem subsystem. See section 0 for more detailed description of each of these parts of the code.

5.1.4.2 Methods – Hardware Control

```
35  void CANRollerSubsystem::RunRoller(double forward, double reverse) {
36    rollerMotor.Set(forward - reverse);
37  }
```

The method RunRoller sets the speed of the roller motor.

5.2 Commands

Commands are what tell the robot when to run the different components that are defined in their subsystems. Commands are "scheduled" for execution, performed, and then removed from the command scheduler. Each command has 4 distinct parts that are performed throughout its lifecycle:

- Initialize: performed when the command is initially scheduled. Anything put in the initialize section of a command will run right before the main body of the command.
- Execute: the main body of the command, which runs once every loop cycle (~20 ms)
- End: performed when the command is removed from the scheduler.
- isFinished: called once per loop cycle after the execute method. isFinished returns true when the condition for exiting the command is met. When isFinished returns true, the end method is called.

As we discussed in section 4, The two ways of writing commands that we focus on in this tutorial are Traditional commands which subclass the WPILib Command class, and command factories, which use command decorators to create commands. Regardless of the way your team chooses to make commands, this is the underlying structure that commands follow.

5.3 Traditional Commands

This subsection will focus on the Traditional command-based framework. Traditional commands are defined in their own classes by subclassing the generic WPILib Command.

5.3.1 DriveCommand.h

The drive command is what tells the kitbot's drive motors to run.

5.3.1.1 Includes, Class Declaration and Constructor

```
#include <frc2/command/Command.h>
#include <frc2/command/CommandHelper.h>

#include <utility>

#include "subsystems/CANDriveSubsystem.h"

class DriveCommand : public frc2::CommandHelper<frc2::Command, DriveCommand> {
 public:
  DriveCommand(CANDriveSubsystem *driveSubsystem,
               std::function<double()> xSpeed,
               std::function<double()> zRotation);
```

The first set of includes is for Command classes from WPILib, for our command class to work properly, we must subclass the WPILib CommandHelper class which references the Command class. The next include is for the "utility" header, this header includes the std::function definition we need for our lambda functions for passing values into our command. The last include is for the subsystem this command uses.

5.3.1.2 Methods – Command State Overrides

```
    void Initialize() override;

    void Execute() override;

    void End(bool interrupted) override;

    bool IsFinished() override;
```

Each command has state methods that dictate the behavior of the robot at different points in the command:

- Initialize: called when the command is initially scheduled.
- Execute: called every loop cycle (20 ms) while the command is running.
- End: called when the command ends.
- IsFinished: controls whether the command has finished running.

5.3.1.3 Class Members

```
26    private:
27     CANDriveSubsystem *driveSubsystem;
28     std::function<double()> xSpeed;
29     std::function<double()> zRotation;
30    };
```

The Drive command has 3 class members:

- The drive subsystem.
- Two lambda functions to pass the speed and rotation from the joysticks to the drive subsystem.

5.3.2 DriveCommand.cpp

5.3.2.1 Includes and Constructor

```
5   #include "commands/DriveCommand.h"
6
7   // Command to drive the robot with joystick inputs
8   DriveCommand::DriveCommand(CANDriveSubsystem *driveSubsystem,
9                              std::function<double()> xSpeed,
10                             std::function<double()> zRotation)
11      : driveSubsystem(driveSubsystem), xSpeed(xSpeed), zRotation(zRotation) {
12    AddRequirements(driveSubsystem);
13  }
```

A command should have an AddRequirements line for any subsystems it controls the outputs (i.e. motors) of. If it is going to be a default command of a subsystem, it should always require this subsystem.

5.3.2.2 Methods – Command State Overrides

```
28  void DriveCommand::Initialize() {}
29
30  void DriveCommand::Execute()
31  {
32      driveSubsystem->ArcadeDrive(xSpeed(), zRotation());
33  }
34
35  void DriveCommand::End(bool interrupted) {}
36
37  bool DriveCommand::IsFinished() { return false; }
```

In the execute method, we call the ArcadeDrive method from the drive subsystem which tells the motors to drive such that the robot moves according to the joystick inputs.

5.3.3 RollerCommand.h

The .h file for this command is very similar to the Drive command. See section 5.3.1 for more detailed description of each of the parts of the code.

5.3.4 RollerCommand.cpp

The first section of this command is very similar to the Drive command. See section 5.3.2 for more detailed description of each of the parts of the code.

5.3.4.1 Methods – Command State Overrides

```
28  void RollerCommand::Initialize() {}
29
30  void RollerCommand::Execute()
31  {
32      rollerSubsystem->RunRoller(forward(), reverse());
33  }
34
35  void RollerCommand::End(bool interrupted) {}
36
37  bool RollerCommand::IsFinished() { return false; }
```

In the execute method of the command, this command calls the RunRoller method of the Roller subsystem in order to set the roller speed.

5.3.5 AutoCommand.h

The first section of this command is very similar to the Drive command. See section 5.3.1 for more detailed description of each of the parts of the code.

5.3.5.1 Includes, Class Declaration and Constructor

```
7   #include <frc/Timer.h>
8   #include <frc2/command/Command.h>
9   #include <frc2/command/CommandHelper.h>
10
11  #include "subsystems/CANDriveSubsystem.h"
12
13  class AutoCommand : public frc2::CommandHelper<frc2::Command, AutoCommand> {
14   public:
15    AutoCommand(CANDriveSubsystem *driveSubsystem);
```

In addition to the includes seen in other commands, this command has an include for the FRC Timer class which this command uses to time the auto maneuver.

5.3.5.2 Methods – Command State Overrides

```
20      void Initialize() override;
21
22      void Execute() override;
23
24      void End(bool interrupted) override;
25
26      bool IsFinished() override;
```

5.3.5.3 Class Members

```
25    private:
26     CANDriveSubsystem *driveSubsystem;
27
28     frc::Timer timer;
29     double seconds = 1;
30    };
```

The timer is used to control how long the robot drives for and the seconds variable is used to define how long that will be. In this simple example the value is hard coded, but you could make the command more flexible by passing in the value instead.

5.3.6 AutoCommand.cpp

The first section of this command is very similar to the Drive command. See section 5.3.2 for more detailed description of each of the parts of the code.

5.3.6.1 Methods – Command State Overrides

```
15  void AutoCommand::Initialize() {
16    // start timer when command is scheduled
17    timer.Restart();
18  }
19
20  void AutoCommand::Execute() {
21    // run the drive train at 50% power forward (called every ~every 20ms)
22    driveSubsystem->ArcadeDrive(0.5, 0.0);
23  }
24
25  void AutoCommand::End(bool interrupted) {
26    // stop the timer and drive train
27    timer.Stop();
28    driveSubsystem->ArcadeDrive(0.0, 0.0);
29  }
30
31  bool AutoCommand::IsFinished() {
32    // the command finishes when the timer exceeds the number of seconds
33    return timer.Get().value() >= seconds;
34  }
```

- Initialize: Start the timer. The Restart function is used in case this command is run more than once without resetting the robot code. If the Timer has not run, Restart will do the same thing as Start, but if it has run Restart will clear it before starting.
- Execute: Sets the drive motors to drive forward.
- End: Stop the timer and stop the drive subsystem.
- IsFinished: Returns true when the timer exceeds 1 second.

5.4 Factory Commands

This subsection will focus on the factory command-based framework. In this structure, commands are defined in their subsystems using WPILib inline commands and decorators and accessed (to bind them) using "factory" methods that create an instance of the command and return it.

5.4.1 Drive Command Factory

5.4.1.1 CANDriveSubsystem.h

With Traditional commands:

```
22    void ArcadeDrive(double xSpeed, double zRotation);
```

With Factory commands:

```
21  frc2::CommandPtr ArcadeDrive(CANDriveSubsystem *driveSubsystem,
22                               std::function<double()> xSpeed,
23                               std::function<double()> zRotation);
```

The ArcadeDrive method is different with Factory command-based code. Rather than returning void, the method returns a command and rather than taking parameters as doubles, it takes functions that return a double.

5.4.1.2 CANDriveSubsystem.cpp

With Traditional commands:

```
64  void CANDriveSubsystem::ArcadeDrive(double xSpeed, double zRotation) {
65    drive.ArcadeDrive(xSpeed, zRotation);
66  }
```

With Factory commands:

```
64  // Command to drive the robot with joystick inputs
65  frc2::CommandPtr CANDriveSubsystem::ArcadeDrive(
66      CANDriveSubsystem *driveSubsystem, std::function<double()> xSpeed,
67      std::function<double()> zRotation) {
68    return frc2::cmd::Run(
69        [this, xSpeed, zRotation] { drive.ArcadeDrive(xSpeed(), zRotation()); },
70        {driveSubsystem});
```

The frc2::cmd::Run() method creates a command which executes everything inside the {}, in this case, every loop cycle (20 ms) it runs the ArcadeDrive method from the DifferentialDrive WPILib class with the inputs being the lambda functions so we can pass a changing value like a joystick input.

5.4.2 Roller Command Factory

5.4.2.1 CANRollerSubsystem.h

With Traditional commands

```
21    void RunRoller(double forward, double reverse);
```

With Factory commands

```
19    frc2::CommandPtr RunRoller(CANRollerSubsystem *rollerSubsystem,
20                               std::function<double()> forward,
21                               std::function<double()> reverse);
```

5.4.2.2 CANRollerSubsystem.cpp

With Traditional commands

```
35  void CANRollerSubsystem::RunRoller(double forward, double reverse) {
36    rollerMotor.Set(forward - reverse);
37  }
```

With Factory commands

```
frc2::CommandPtr CANRollerSubsystem::RunRoller(
    CANRollerSubsystem *rollerSubsystem, std::function<double()> forward,
    std::function<double()> reverse) {
  return frc2::cmd::Run(
      [this, forward, reverse] { rollerMotor.Set(forward() - reverse()); },
      {rollerSubsystem});
```

5.4.3 Autos

The Autos file is an example of a "Static Command Factory". Your program should never create an Autos object (as shown by the constructor simply printing an error message) instead you call class methods statically using autos::ExampleAuto() type syntax. This structure is one of the ways to define complex groups that involve multiple subsystems (though our example here is not complex and requires only a single subsystem). The .h file for this class is pretty simple so it's not depicted here.

```
frc2::CommandPtr autos::ExampleAuto(CANDriveSubsystem *driveSubsystem) {
  return frc2::cmd::Sequence(
      driveSubsystem
          ->ArcadeDrive(
              driveSubsystem, []() { return 0.5; }, []() { return 0.0; })
          .WithTimeout(1.0_s),
      driveSubsystem->ArcadeDrive(
          driveSubsystem, []() { return 0.0; }, []() { return 0.0; }));
}
```

Our example file only has a single autonomous routine to get. You could easily extend this pattern by adding additional methods to define more autonomous routines and you could select between them using a SendableChooser on the dashboard.

This simple autonomous routine instructs the robot to drive forwards for 1 second at 50% power by using the WithTimeout decorator to set a timeout of 1 second on the driving command. It uses the AndThen decorator to tell the robot to stop moving after the first command is complete. The different types of command compositions that are built-in via decorators and factory methods are described on the Command Compositions page.

5.5 Constants

This class contains named constants used elsewhere in the code. Subclasses are used to organize the constants into distinct groups, in this case by subsystem. The provided constant names should pretty clearly describe what each is for.

5.6 Robot.cpp

This file is identical to the default Command-Based template. You can find a description of the elements in the Robot class in the Structuring a Command-Based Robot Project article.

5.7 RobotContainer

The RobotContainer class is where instances of the robot subsystems and controllers are declared and where default commands and mappings of buttons to commands are defined.

5.7.1.1 RobotContainer.h – Class definition, Constructor and Auto method

```
25  class RobotContainer {
26   public:
27    RobotContainer();
28
29    frc2::CommandPtr GetAutonomousCommand();
```

This section of code declares the class and constructor and declares a public method to get the AutonomousCommand that should be used. This method is called from Robot.cpp when the robot is switched into Auto mode to determine what command to run. See Section 6.4 for more information on developing Autonomous Modes.

The constructor definition in RobotContainer.cpp contains a call to configureBindings() which we will cover below. This method is used to set up button bindings. It also sets the default command for the drivetrain subsystem. You are welcome to put this code in the ConfigureBindings method if that makes more sense to you, or even add a new method just for setting up default commands! In this tutorial, we will put the default commands inside the ConfigureBindings method.

5.7.1.2 RobotContainer.h - Member Variables

```
28  private:
29   frc2::CommandXboxController driverController{
30       OperatorConstants::DRIVER_CONTROLLER_PORT};
31   frc2::CommandXboxController operatorController{
32       OperatorConstants::OPERATOR_CONTROLLER_PORT};
33
34   CANDriveSubsystem driveSubsystem;
35   CANRollerSubsystem rollerSubsystem;
```

The section defines the class member variables. For RobotContainer this generally includes all of your subsystems and control devices. This code uses the CommandXboxController to represent the gamepads as it contains a number of helper methods that make it much easier to connect commands to buttons. We define these as private. If things outside RobotContainer need access to them (like commands) they should accept a pointer to them as a parameter or methods should be added to robot container to expose the necessary information (like a joystick axis).

5.7.1.3 RobotContainer.cpp - Configure Bindings Method

```
18  void RobotContainer::ConfigureBindings() {
19    // Set the default command for the drivetrain to use the DriveCommand to drive
20    // the robot with the left Y and right X axes of the gamepad
21    driveSubsystem.SetDefaultCommand(driveSubsystem.ArcadeDrive(
22        &driveSubsystem, [this] { return -driverController.GetLeftY(); },
23        [this] { return -driverController.GetRightX(); }));
24
25    // Set the default command for the roller to be analog control mapped to the
26    // trigger axes of the 2nd gamepad
27    rollerSubsystem.SetDefaultCommand(rollerSubsystem.RunRoller(
28        &rollerSubsystem,
29        [this] { return operatorController.GetRightTriggerAxis(); },
30        [this] { return operatorController.GetLeftTriggerAxis(); }));
```

We want a command to run on our drivetrain to allow us to drive the robot with joysticks whenever we don't have some other command using the drivetrain (like the ExampleAuto command). To do this we use the SetDefaultCommand() method of the subsystem. This sets the command that will run whenever the Scheduler sees nothing else running on that subsystem.

In this code, the method we want to call is the ArcadeDrive method of the drivetrain subsystem, which returns a command object that runs the drivetrain. For the TraditionalCommands project, these lines will look a little different as they will instead use the DriveCommand class that has been created. For the forward/back movement we pass in the value from the Y-axis (vertical) of the left stick of the controller, but we negate it. This is because joysticks generally define pushing the stick away from you as negative and pulling the stick towards you as positive (a result of the original use being flight simulators). We want pushing the stick away from us to drive the robot forward, so we negate the value. Similar for the turning value where we negate the X-axis (horizontal) of the right stick of the controller. The joystick considers pushing this to the right as a positive value, but the WPILib classes consider clockwise rotation (what would be expected when pushing the joystick right) as negative.

A similar binding is used to set the default command for the Roller subsystem using the operator controller triggers as the values.

```
32    // While the A button of the 2nd gamepad is held, run the roller at a fixed
33    // speed
34    operatorController.A().WhileTrue(rollerSubsystem.RunRoller(
35        &rollerSubsystem, [this] { return RollerConstants::ROLLER_EJECT_SPEED; },
36        [this] { return 0; }));
```

Finally, we bind a command to the A button of the operator controller. The CommandXboxController class contains methods for each button which return "Trigger" objects. These "Trigger" objects then have further methods that narrow down the behavior we want to control the command such as

toggles, initiating on change, or running only while the Trigger is true or false. In this instance we use "WhileTrue()" to have our command run while the button is being held and stop when it is released. Because our RunRoller Command takes std::function types as it's parameters (so that it can retrieve changing joystick values in the other usage of it), we need to use a lambda function to pass the constant values.

6 Making Changes

This section details some common possible changes you may want to make to the KitBot code and provides some references for how to approach making those changes.

6.1 Changing buttons for actions

One of the easiest changes to make to Command-Based robot code is to switch what buttons or button behaviors control a command. Other than the auto command, commands used in the 2025 KitBot do not end (isFinished always returns false) so they should generally only be used with the whileTrue() behavior, but changing which buttons they map to can be done very simply.

The button mappings in the example code are done near the end of the configureBindings() method inside the RobotContainer file. The bindings used for this project are made using the helper methods of the CommandXboxController class. These helper methods exist for each button on the controller and return a Trigger object which has further methods that can be used to specify a behavior for the binding.

As provided the code connects the **A** button to the fixed power. To change this, simply change the A() helper method to the method for any of the other buttons! You can see all of the available options by looking through the CommandXboxController Javadoc for methods which take no parameter and return a Trigger object.

For example, to change the intake command from the A button to the X button, simply replace the A() with X()

```
//Before
operatorController.A().WhileTrue(
    RollerCommand(
        &rollerSubsystem,
        [this] { return RollerConstants::ROLLER_MOTOR_EJECT_SPEED; },
        [this] { return 0; })
        .WithName("Fixed speed"));
//After
operatorController.X().WhileTrue(
    RollerCommand(
        &rollerSubsystem,
        [this] { return RollerConstants::ROLLER_MOTOR_EJECT_SPEED; },
        [this] { return 0; })
        .WithName("Fixed speed"));
```

6.2 Changing Drive Axis Behavior

Another easy change to make is to modify which axes of the controller are used as which part of the robot driving and how. The provided code does this mapping when setting up the drivetrain default command at the top of ConfigureBindings() in RobotContainer.

The example code uses the Y-axis of the left stick to drive forward and back and the X-axis of the right stick to rotate. These can easily be swapped to the opposite sticks or move just one so they are on the same stick! To review the available options, look for methods that return a **double** in the CommandXboxController API Docs. To make this type of modification, locate the method call you wish to change, such as GetLeftY(), and replace it with the new desired method, such as GetRightY()

Example: changing the rotation driving to the left stick X-axis and leaving the translation on the left Y-axis

```
19    driveSubsystem.SetDefaultCommand(driveSubsystem.ArcadeDrive(
20        &driveSubsystem,
21        [this]
22        { return -driverController.GetLeftY(); },
23        [this]
24        { return driverController.GetLeftX(); }));
```

You can also modify the axis values. One common modification is to cube the values. This preserves the sign of the value (positive stays postive, negative stays negative) and the maximum value (doesn't reduce the max speed of the robot) while providing less sensitivity at low inputs, potentially allowing for more precise control at low speeds. To make this type of modification, you can apply the modification to the axis where it's being captured.

Example: changing only the rotation axis to be cubed:

```
19    driveSubsystem.SetDefaultCommand(driveSubsystem.ArcadeDrive(
20        &driveSubsystem,
21        [this]
22        { return -driverController.GetLeftY(); },
23        [this]
24        { return pow(driverController.GetLeftX(), 3); }));
```

Another common modification is to scale the values down by default, but allow for the maximum value if a button is pressed (turbo mode). This type of modification can also be done at the point of capture, though as complexity grows, you may wish to shift from an inline command definition to a different type where you can define the command behavior more clearly.

Example: Scale the forward-back driving by 50% unless the right bumper is pressed

```
19    driveSubsystem.SetDefaultCommand(driveSubsystem.ArcadeDrive(
20        &driveSubsystem,
21        [this]
22        { return -driverController.GetLeftY() * driverController.GetLeftBumper()?1:0.5},
23        [this]
24        { return driverController.GetLeftX(); }));
```

This example uses two specific things that warrant explanation. The first one is that it uses GetRightBumper method on the driver controller. This returns the Boolean value of the button rather than a Trigger object associated with the button like RightBumper() would.

The other thing this example uses is the ? operator, called the ternary operator. This operator allows us to write a simple "if" statement in a very compact way, if the right bumper is pressed, we multiply by 1, if not, by 0.5.

6.3 Changing Drive Type

The last likely change we will cover is changing from Arcade Drive to Tank Drive. Unlike Arcade drive which maps one axis to rotation and one to forward/back, Tank drive maps one axis (generally the Y-axis) to each side of a differential drivetrain. To make this change, you'll have to reach beyond RobotContainer as the provided drivetrain subsystems don't expose a tank drive method. In the appropriate drivetrain subsystem (PWMDriveSubsystem or CANDriveSubsystem) make a new method called TankDrive(). Make sure to add it to both the .h and .cpp files! This method should look a lot like the ArcadeDrive method. Then, modify the default command mapping in RobotContainer to use this new method with the appropriate joystick axis.

Example:

```
22    frc2::CommandPtr TankDrive(
23        PWMDriveSubsystem *driveSubsystem,
24        std::function<double()> xSpeed,
25        std::function<double()> zRotation);
```

```
29  frc2::CommandPtr PWMDriveSubsystem::TankDrive(PWMDriveSubsystem *driveSubsystem, std::function<double()> xSpeed, std::function<double()> zRotation)
30  {
31    return frc2::cmd::Run([this, xSpeed, zRotation]
32                          { drive.TankDrive(xSpeed(),
33                                            zRotation()); },
34                          {driveSubsystem});
35  }
```

```
19    driveSubsystem.SetDefaultCommand(driveSubsystem.TankDrive(
20        &driveSubsystem,
21        [this]
22        { return -driverController.GetLeftY(); },
23        [this]
24        { return -driverController.GetRightY(); }));
```

6.4 Developing Autonomous Routines

The provided code contains a very basic autonomous mode that drives forwards at ½ power for 1 second. Additional autonomous modes can be developed, see the Hatchbot Inlined example or Hatchbot Traditional example for a project with multiple autos.

It's common (but definitely not required!) to have multiple autonomous routines that you may wish to run based on different starting locations or strategies. If you pursue this, the most common way to choose between them for each match is to select between them using a SendableChooser on the dashboard.